Question 1 [15 Marks]

a. Briefly explain the techniques below. Be sure the difference between the two problem-solving strategies in each pair is clear. [6]

1. *divide-and-conquer* and *decrease-and-conquer*
2. *space-and-time tradeoff* and *problem reduction*
3. *dynamic programming ("DP")* and *DP with memory functions*

div-conq: solve problem by combining solutions of smaller subproblems

decrease-conq: solve problem by solving smaller instance of that same problem

space-time: solve problem faster by using more space (or vice versa, if space is the problem)

prob reduction: solve problem A by transforming it to an instance of another problem B

DP: solve problem by combining solutions of overlapping smaller subproblems

DP with mem func: as above, but proceed top down instead of bottom up

b. State whether each of the following is true or false: [5]

1. $\Theta(n + \log n) = \Theta(n)$
2. $O(n \log n) = O(n)$
3. if $p \in O(n \log n)$ then $p \in O(n^2)$
4. if $p \in \Theta(n \log n)$ then $p \in \Theta(n^2)$
5. if $p \in \Omega(n \log n)$ then $p \in \Omega(n)$

1. **T** 2. **F** 3. **T** 4. **F** 5. **T**

c. Exponentiation function a^n calculates a to the power n. Show how the problem of finding a^n is tackled using each of the following techniques (1 line each): [2]

1. brute force
2. decrease by a constant
3. decrease by a constant factor
4. divide and conquer

brute force: a * a * … * a (n times)

decrease by constant: $a^{n-1} * a$

decrease by constant factor: $(a^{n/2})^2$

divide and conquer: $a^{n/2} * a^{n/2}$

d. Use the Master Theorem to find the complexity of *waat* below, **<u>showing your working</u>**.

waat (x) { if x < 3 return 'a' else return waat(2x/3) ++ waat(2x/3) } // ++ is concatenate

Master Theorem: If $T(n) = aT(n/b) + f(n)$ then

$T(n) \in \Theta(n^d)$ if $a < b^d$

$T(n) \in \Theta(n^d \log n)$ if $a = b^d$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$ [2]

a is 2, b is 3/2, d is 0

so $a > b^d$ so $\Theta(n^{\log_{3/2} 2})$

Question 2 [17 Marks]

a. Consider searching for the first occurrence of the octal pattern 061606 in octal text 02040606061606

1. Show the shift table that would be used if Horspool's algorithm is applied. [2]
2. Show the good-suffix table that Boyer-Moore would use for this search. [4]
3. The search starts by aligning the pattern with the first character of the text. Where will it move the pattern to next (i.e. for the 2nd alignment) using: [2]
   i. Horspool's algorithm
   ii. the Boyer-Moore algorithm
4. Can the Boyer-Moore algorithm ever have a good-suffix shift table with elements S[j] and S[k] such that j < k but S[j] > S[k]? If no, explain why not. If yes, give an example of such a search pattern. [1]
5. Which algorithm is more efficient: Horspool or Boyer-Moore? [1]

1. Horspool shift table:

0	1	2	3	4	5	6	7
1	**3**	6	6	6	6	**2**	6

2. Good-suffix table:

1	2	3	4	5
2	**4**	**4**	**4**	**4**

3i Horspool shifts up <whatever number they had in HorspoolShiftTable[6] above> or **2**

3ii Boyer-Moore shifts up <whatever number they had in GoodSuffixTable[2] above> or **4**

4. **Yes,** many examples e.g. **anon** or **teepee** or c**eded** other pattern where suffix occurs earlier in string - but not if that pattern's first char/s are also how it ends eg **baba** or **ar**tisanal-b**ar**

5. **Boyer-Moore** (is more efficient).

1. Show the first 2 arrays (only!) that would be generated by Warshall's transitive closure algorithm (i.e. after initialising, and after the 1st iteration of the algorithm). [4]

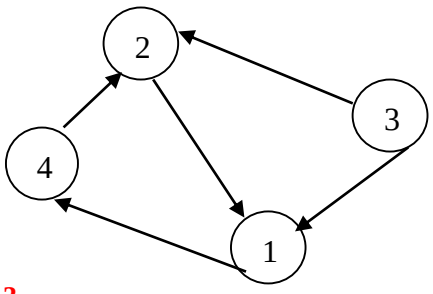
0	0	0	**1**
1	0	0	0
1	**1**	0	0
0	**1**	0	0

0	0	0	1
1	0	0	**1**
1	1	0	**1**
0	1	0	0

or blue below if had rows as "to" and columns as "from"

0	**1**	**1**	0
0	0	**1**	**1**
0	0	0	0
1	0	0	0

0	1	1	0
0	0	1	1
0	0	0	0
1	**1**	**1**	0



2. What is the time complexity of Warshall's algorithm? n^3 [1]

3. What is the space complexity of Warshall's algorithm? n^3 **(but I'll accept n^2 too)** [1]

4. Can its space complexity be improved? If yes state how, if no state why not. [1]

Yes, can use the same one array on each iteration and just overwrite values instead of making an extra new array each time.

Question 3 [8 Marks]

a. You are in a race to find the treasure on a map. Each time you guess a point by giving its X and Y co-ordinates, you are told "YES!" or given the direction (from your guess) in which the treasure is. This direction can be North, South, East, West, North-East, North-West, South-East or South-West. The X values of the map range from -N in the West to +N in the East; the Y values of the map range from -N in the South to +N in the North.

1. Explain briefly how a brute force approach would go about solving this problem. [**1**]
2. Explain briefly another algorithm that can solve this problem more efficiently than your brute force one. [**3**]
3. What is the complexity of your brute force algorithm? [1]
4. What is the complexity of your other (more efficient) algorithm? [1]

1. Guess each possible point using 2 nested loops that both go from –N to N until get "YES!"

2. Use **binary search (or decrease and conquer).**

Initialise:Top=N, Botm=(-N), Lef=(-N), Right=N.

Repeatedly **guess** X=Lef + (Right-Lef)/2, Y=Botm + (Top-Botm)/2 and reduce the search area based on reply.

E.g. if reply is "East" set Lef = X+1 and Top=Botm=Y; - - - ; if it's "North" set Botm=Y+1 and Lef=Right=X; - - -;

if it is "South-East" set Lef=X-1 and Top = Y-1.

If reply is "YES!" exit and display X,Y as location of treasure.

1 mark for underlined blue stated or described. **½ point for each underlined red aspect.**

3. N^2

4. log N

b. The *Dutch Flag Problem* is to arrange an array of characters B, O, W (blue, orange, white) so that all the O's come first, then all the W's next, and finally all the B's last. Briefly describe a linear time algorithm for this problem. [2]

Create 3-element array X initialised with zeroes: X[0] is number of O's found, X[1] is number of W's found, X[2] is number of B's found. Do linear scan of array of characters incrementing value of corresponding X element each time. Finally scan X[j] for j=0 to 2 and output X[0] O's, then X[1] W's and lastly X[2] B's.

If just says use distribution sort, or just says use sorting by counting, then only 1 mark not 2.